Notación Big-O

Santiago Javier Proaño Yépez

<2026-10-13 mar>

Contents

1 La notación Big-O **1**
1.1 Propiedades de la notación 1
1.1.1 Polinomios 1
1.1.2 Regla de las constantes multiplicativas 2
1.1.3 Regla de la suma 2
1.1.4 Regla de la multiplicación 2
1.2 Bucles ATTACH 3
1.2.1 Independientes anidados ATTACH 3
1.2.2 Bucles dependientes anidados ATTACH 4
1.3 Secuencias de declaraciones ATTACH 4
1.4 Tasas de crecimiento ATTACH 5

1 La notación Big-O

Como vimos define que tan rápido crece una función respecto a un techo. Es decir, que big O demuestra que una funcion no va a ser peor que algo, o sea que analiza el peor de los casos para cada función. Debido a esto es que se usa esta notacion para representar la complejidad del algoritmo y como este va escalando conforme escala la entrada.

1.1 Propiedades de la notación

1.1.1 Polinomios

Gracias a la definición de la notación para determinar más fácilmente la constante dentro de una funcion f(n) que es un polinomio lo que podemos hacer es remplazar cada termino inferior al grado con dicho grado y asi obtenemos la constante y el valor de x en O(x). Por ejemplo

$$5n^4 + 3n^3 + 2n^2 + 4n + 1 \leq cg(n)$$
$$5n^4 + 3n^3 + 2n^2 + 4n + 1 \leq 5n^4 + 3n^4 + 2n^2 + 4n^2 + n^4$$
$$5n^4 + 3n^3 + 2n^2 + 4n + 1 \leq (5 + 3 + 2 + 4 + 1)n^4$$
$$5n^4 + 3n^3 + 2n^2 + 4n + 1 \leq 15n^4$$

Por lo tanto la constante C en este sería 15 y el g(n) sería n^4 entonces por definición

$$O(g(n)) = O(n^4)$$

El grado más alto del polinomio es la tasa de crecimiento asintótico

1.1.2 Regla de las constantes multiplicativas

Se ignoran los factores constantes

$$O(cg(n)) = O(g(n)) \text{ para una constante } C > 0$$

1.1.3 Regla de la suma

Se ignoran los factores más pequeños

$$\text{si } O(q(n) < g(n))$$
$$\text{entonces } O(q(n) + g(n)) = O(g(n))$$

1.1.4 Regla de la multiplicación

$$O(h(n)) \times O(g(n)) = O(h(n) \times g(n))$$

Por ejemplo Luego de multiplicar nos que da la expresión

$$O((n^3 + 2n^2 + 3n \log n + 7)(8n^2 + 5n + 2) = O(n^5)$$

1.2 Bucles ATTACH

La complejidad de un bucle esta determinada por el número de iteraciones en el ciclo por la complejidad del cuerpo del ciclo

```
for (int i = 0; i < n; i++)
     sum = sum - i;
```

O(n)

```
for (int i = 0; i < n * n; i++)
      sum = sum + i;
```

$O(n^2)$

```
int i=1;
while (i < n) {
      sum = sum + i;
      i = i*2
}
```

O(log n)

```
for(int i = 0; i < 100000; i++)
      sum = sum + i;
```

O(1)

1.2.1 Independientes anidados ATTACH

Se determina por la complejidad del bucle interno * la complejidad del bucle externo

```
int sum = 0;
for(int i = 0; i < n; i++)
    for(int j = 0; j < n; j++)
        sum += i * j ;
```

$O(n^2)$

```
int i = 1, j;
while(i <= n) {
   j = 1;
   while(j <= n){
      statements of constant complexity
      j = j*2;
   }
   i = i+1;
}
```

O(n log n)

1.2.2 Bucles dependientes anidados ATTACH

La complejidad del externo afecta al interno

jemplos:

```
int sum = 0;
for(int i = 1; i <= n; i++)
    for(int j = 1; j <= i; j++)
        sum += i * j ;
```

El número de repeticiones del bucle interno es : 1 + 2 + 3 + . . . + n = n(n+2)/2

Por lo tanto, el segmento es **$O(n^2)$**

```
int sum = 0;
for(int i = 1; i <= n; i++)
    for(int j = i; j < 0; j++)
        sum += i * j ;
```

El número de repeticiones del bucle interno es: 0
El bucle externo itera n veces.
Por lo tanto, el segmento es **O(n)**

1.3 Secuencias de declaraciones ATTACH

Básicamente aquí se hace la regla de la suma con todas las complejidades de todas las declaraciones por ejemplo:

```
for (int j = 0; j < n * n; j++)
   sum = sum + j;

for (int k = 0; k < n; k++)
   sum = sum - 1;

System.out.print("sum is now " + sum);
```

$O(n^2)$ $O(n)$ $O(1)$

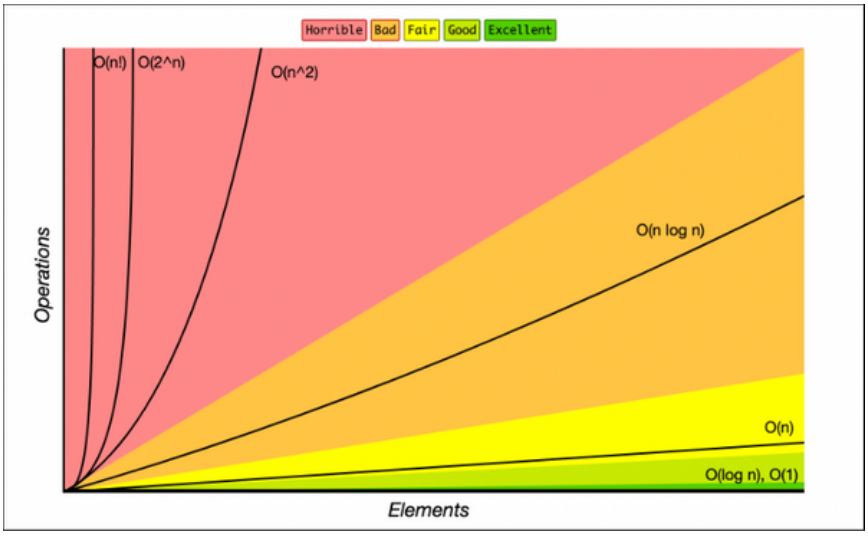
Por tanto, la complejidad es: $O(n^2) + O(n) + O(1) = O(n^2)$

1.4 Tasas de crecimiento ATTACH

Dependiendo la complejidad por big o, las tasas de crecimiento se verian asi

	constant	*logarithmic*	*linear*	*N-log-N*	*quadratic*	*cubic*	*exponential*
n	$O(1)$	$O(\log n)$	$O(n)$	$O(n \log n)$	$O(n^2)$	$O(n^3)$	$O(2^n)$
1	1	1	1	1	1	1	2
2	1	1	2	2	4	8	4
4	1	2	4	8	16	64	16
8	1	3	8	24	64	512	256
16	1	4	16	64	256	4,096	65536
32	1	5	32	160	1,024	32,768	4,294,967,296
64	1	6	64	384	4,069	262,144	1.84×10^{19}

Sin embargo para tener una noción de cual es peor o mejor esta mejor esta gráfica


Horrible
Bad
Fair
Good
Excellent
O(n!)
O(2^n)
O(n^2)
O(n log n)
O(n)
O(log n), O(1)
Operations
Elements